

Chapter 6: Command-line Options and typed variables.

Command line options:

Recap→ Positional parameters→ Shell used to store the command-line arguments to a shell script or function to run.

The form **command [-options]args**, meaning that there can be 0 or more options. Here, the command is \$0, the option is \$1, args are \$2. So the order of each word is marked.

Shift: a command used to manipulate positional parameters by shifting them to the left.

So when shift is called, \$1 is discarded and \$2 becomes \$1, \$3 becomes \$2 and so on. The number of positional parameters decreases by 1.

Just using shift:

```
echo "All arguments: $@"
while [ $# -gt 0 ]; do
    echo "processing: $1"
    shift
done
```

```
● gayat@GSP-HP:~/SummerInternship$ ./echo.sh 1 2 3 4 5 6 7 8 9
All arguments: 1 2 3 4 5 6 7 8 9
processing: 1
processing: 2
processing: 3
processing: 4
processing: 5
processing: 6
processing: 7
processing: 8
processing: 9
```

Expression	Meaning	Quote behavior
\$@	All arguments, as separate strings	Quoted: "hello" "world"
\$*	All arguments as one string	Quoted: "hello world"

\$# in Bash means:

"The number of positional parameters (arguments)" passed to a script or function.

```
echo "All arguments: $@"
while [ $# -gt 0 ]; do
    echo "processing: $1"
    if [ $# -ge 2 ]; then
        shift 2
    else
        shift 1
    fi
done
```

```
● gayat@GSP-HP:~/SummerInternship$ chmod +x echo.sh
● gayat@GSP-HP:~/SummerInternship$ ./echo.sh 1 2 3 4 5 6 7 8 9
All arguments: 1 2 3 4 5 6 7 8 9
processing: 1
processing: 3
processing: 5
processing: 7
processing: 9
```

So basically, the first one is kept as reference and then shift 2 jumps to 3rd, and from there to 5th.

```
while [ $# -gt 0 ]; do      # While there are still positional parameters left (i.e., $# > 0), continue looping
    echo "$1 : $2"         # Print the current key ($1) and its value ($2)
    shift 2                # Remove $1 and $2 from the list, so $3 becomes $1, $4 becomes $2, and so on
done
```

```
● gayat@GSP-HP:~/SummerInternship$ chmod +x echo.sh
● gayat@GSP-HP:~/SummerInternship$ ./echo.sh name Alice age 21 country India
name : Alice
age : 21
country : India
```

```
echo "All args: $@"
echo "Before shift, first arg: $1"
shift
echo "After shift, first arg:$1"
```

Essentially, the first argument you provided is discarded, and the second argument becomes the first after the shift.

```
● gayat@GSP-HP:~/SummerInternship$ ./echo.sh apple orange banana cherry
All args: apple orange banana cherry
Before shift, first arg: apple
After shift, first arg:orange
```

```
while [ "$#" -gt 0 ]; do
echo "Processing: $1"
shift
done
```

This script prints each argument and shifts the rest left and continues until no arguments are left. It means, while the number of arguments is greater than 0, do print them and shift them each time, printing in each line..

```
● gayat@GSP-HP:~/SummerInternship$ ./echo.sh apple orange banana cherry
Processing: apple
Processing: orange
Processing: banana
Processing: cherry
```

```
while [ "$#" -gt 0 ]; do
echo "Processing: $1"
shift 2
done
```

This script prints the first and 3rd arguments only, as we put shift 2, so from 1st argument, shift 2 means the shell jumps to 3rd argument and since we don't have a 5th argument and we had a while loop, telling until the number of arguments is greater than 0, it stopped.

```
● gayat@GSP-HP:~/SummerInternship$ ./echo.sh apple orange banana cherry
Processing: apple
Processing: banana
```

```
while [ "$#" -gt 0 ]; do
echo "The number of arguments: $#"
```

```
echo "The arguemnt: $1"
```

```
shift
```

```
done
```

Here, I tried to print the number of arguments and the argument using the shift and while loop.

```
● gayat@GSP-HP:~/SummerInternship$ chmod +x echo.sh
● gayat@GSP-HP:~/SummerInternship$ ./echo.sh apple orange banana cherry
The number of arguments: 4
The arguemnt: apple
The number of arguments: 3
The arguemnt: orange
The number of arguments: 2
The arguemnt: banana
The number of arguments: 1
The arguemnt: cherry
```

```

shift 2
while [ "$#" -gt 1 ]; do
echo "$1"
shift
done

```

Here, I tried to skip the first 2 arg using skip 2 and then I made a while loop and shift to print the whole thing left.

```

● gayat@GSP-HP:~/SummerInternship$ chmod +x echo.sh
● gayat@GSP-HP:~/SummerInternship$ ./echo.sh a b c d e
c
d
○ gayat@GSP-HP:~/SummerInternship$ █

```

Notes:

In test expressions [], these are logical operators

Flag	Meaning
-a	AND (use && in [[]] instead)
-o	OR (use `
-b	File is a block device
-c	File is a character device
-d	File is a directory
-e	File exists
-f	File is a regular file
-r	File is readable
-w	File is writable
-x	File is executable

getopts:

A built-in command to use as the condition of the while in an option-processing loop. (Helps to use options and their arguments cleanly, by supporting short options, options with arguments (like -f file), grouped options (like -abc), and even long options.)

Syntax for getopt → getopt optstring parameters

Getopt takes 2 arguments—1st one with letters and a colon.

Optstring→string of valid option characters, if an option is separated by : , it means it needs an argument to work with it.

Parameters→ the command line arguments

What getopt does is that it picks up options off the command line and assigns each one to a variable which is the parameter/2nd argument on the line

```
while getopt ":ab:c" opt; do
  case "$opt" in
    "a")    echo "Option -a triggered"
    ;;
    "c")    echo "Option -c triggered"
    ;;
    "b")    echo "Option -b with an argument: $OPTARG"
    ;;
    \?)    echo "Invalid options"
    ;;
    :)    echo "Missing argument"
    ;;
    esac
done
```

So, what we did is that we analyzed a string for options and used a case loop to check it and once we ran the code with some options, it gave the output.

```
while getopt ":ab:c" opt; do
  case "$opt" in
    "a")    echo "Option -a triggered"
    ;;
    "c")    echo "Option -c triggered"
    ;;
    "b")    echo "Option -b with an argument: $OPTARG"
    ;;
    \?)    echo "Invalid options"
    ;;
    :)    echo "Missing argument"
    ;;
    esac
done
```

An initial colon causes getopt not to print an error message when the user gives an invalid option.

The parameters are the second arguments, and they are the name of a variable that will hold each option letter (without any leading dash) as it is processed.

If an option takes an argument, the argument is stored in the variable OPTARG

```
while getopts ":n:" opt; do
    case "$opt" in
        n)
            echo "Hello, $OPTARG!"
            ;;
    esac
done
```

```
gayat@GSP-HP:~/SummerInternship$ ./echo.sh -n Gayathri
Hello, Gayathri!
gayat@GSP-HP:~/SummerInternship$
```

```
while getopts "hn:" opt ; do
    case "$opt" in
        h) echo "Usage <option>"
            echo " h Show Help"
            ;;
        n) echo "Hello, $OPTARG!"
            ;;
    esac
done
```

```
gayat@GSP-HP:~/SummerInternship$ ./echo.sh -h -n Gayathri
Usage <option>
 h Show Help
Hello, Gayathri!
```

TYPED VARIABLES:

Variables can be typed or made into a type, like integers. You can set these attributes by declaring them using built-in options.

Option	Meaning
-a	The variables are treated as arrays
-f	Use function names only
-F	Display function names without definitions
-i	The variables are treated as integers

Option	Meaning
-r	Makes the variables read-only
-x	Marks the variables for export via the environment

Declaring a variable is done by typing ***declare*** then the options.

Just typing declare will give all the values of variables in the environment. -F is such an option which limits the display with just the function name.

-i is used to give a variable an integer value.

-x used to export a variable to outside the current shell.

-r can't be unset and variables declared in a function are local to that function, just like using local to declare them.

```
gayat@GSP-HP:~/SummerInternship$ declare -F
declare -f __expand_tilde_by_ref
declare -f __get_cword_at_cursor_by_ref
declare -f __git_eread
declare -f __git_ps1
declare -f __git_ps1_colorize_gitstring
declare -f __git_ps1_show_upstream
declare -f __git_sequencer_status
declare -f __load_completion
declare -f __ltrim_colon_completions
declare -f __parse_options
declare -f __reassemble_comp_words_by_ref
declare -f __updateEnvCache
declare -f __updateEnvCacheAA
declare -f __vsc_command_complete
declare -f __vsc_command_output_start
declare -f __vsc_continuation_end
declare -f __vsc_continuation_start
declare -f __vsc_escape_value
declare -f __vsc_escape_value_fast
declare -f __vsc_get_trap
```

```
gayat@GSP-HP:~/SummerInternship$ declare -i a=45 b=90
gayat@GSP-HP:~/SummerInternship$ declare -i result
gayat@GSP-HP:~/SummerInternship$ result=a+b
gayat@GSP-HP:~/SummerInternship$ echo $result
135
```

Integer Variables and Arithmetic:

Shell takes words in `$(())` as arithmetic expressions. Variables in arithmetic expressions do not need to be preceded by dollar signs, though it is not wrong to do so.

When in doubt, enclose a string in single quotes, unless it contains tildes or any expression involving a dollar sign, in which case you should use double quotes

```
● gayat@GSP-HP:~/SummerInternship$ echo "Only $(( 365 - $(date +%j)) / 7 )) weeks until new year"
Only 28 weeks until new year
```

Arithmetic expression feature, like `getopts` is built into `bash`'s syntax.

Arithmetic Operators:

Operator	Meaning
++	Increment by one (prefix and postfix)
—	Decrement by one (prefix and postfix)

```
● gayat@GSP-HP:~/SummerInternship$ echo $x
46
● gayat@GSP-HP:~/SummerInternship$ echo $((x++))
46
● gayat@GSP-HP:~/SummerInternship$ echo $((++x))
48
```

Operator	Meaning
+	Plus
-	Minus
*	Multiplication
/	Division (with truncation)
%	Remainder
**	Exponentiation ^[10]
<<	Bit-shift left
>>	Bit-shift right
&	Bitwise and

The thing is, `x++` and `++x` both add 1 to the value, but the former adds it and shows it when printed with `echo`, while the latter shows it in the next line itself.

$x \ll n = x \times 2^n \rightarrow$ Bit-shift left

$x \gg n = x \div 2^n$ (integer division)

$\rightarrow \rightarrow$ Bit-shift right
 $\rightarrow$

Operator	Meaning
	Bitwise or
~	Bitwise not
!	Logical not
^	Bitwise exclusive or
,	Sequential evaluation

Relational Operators:

Operator	Meaning
<	Less than

>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
!=	Not equal to
&&	Logical and
	Logical or

```

echo "Enter the value for a:"
read a
echo "Enter the value for b:"
read b

if (( a > b )); then
    echo "$a is greater than $b"
elif (( a < b )); then
    echo "$a is less than $b"
else
    echo "$a and $b are equal, they are the same value, you dumbass!"
fi

```

```

● gayat@GSP-HP:~/SummerInternship$ ./echo.sh
Enter the value for a:
56
Enter the value for b:
56
56 and 56 are equal, they are the same value, you dumbass!
○ gayat@GSP-HP:~/SummerInternship$ █

```

ARITHMETIC VARIABLES AND ASSIGNMENT:

Just like declaring a variable, you can assign an integer/ arithmetic expression to a variable using the command *let*.

```

gayat@GSP-HP:~$ let x=56789
gayat@GSP-HP:~$ echo $x
56789

```

let doesn't create a variable of type integer; it only causes the expression following the assignment to be interpreted as an arithmetic one.

Note that you assign the arithmetic expression to a variable and when you run it, you get the output result to that arithmetic expression.

```

gayat@GSP-HP:~$ let x=67
gayat@GSP-HP:~$ echo $x
67
gayat@GSP-HP:~$ let x=1+5
gayat@GSP-HP:~$ echo $x
6
gayat@GSP-HP:~$ let x='1+5'
gayat@GSP-HP:~$ echo $x
6
gayat@GSP-HP:~$ let y='2+3*5'
gayat@GSP-HP:~$ echo $y
17
gayat@GSP-HP:~$ let y='(2+3)*5'
gayat@GSP-HP:~$ echo $y
25
gayat@GSP-HP:~$ let x='17%3'
gayat@GSP-HP:~$ echo $x
2
gayat@GSP-HP:~$ let x=17%3
gayat@GSP-HP:~$ echo $x
2

```

```

gayat@GSP-HP:~$ let x=17/3
gayat@GSP-HP:~$ echo $x
5
gayat@GSP-HP:~$ let a='1<<4'
gayat@GSP-HP:~$ echo $a
16
gayat@GSP-HP:~$ let a='48>>3'
gayat@GSP-HP:~$ echo $a
6
gayat@GSP-HP:~$ let g='17&3'
gayat@GSP-HP:~$ echo $g
1
gayat@GSP-HP:~$ let g='17|3'
gayat@GSP-HP:~$ echo $g
19
gayat@GSP-HP:~$ let g='17^3'
gayat@GSP-HP:~$ echo $g
18

```

```

let a=5      # 0101
let b=3      # 0011

let and=a&b
let or=a|b
let xor=a^b

echo "AND: $and" # Output: 1
echo "OR : $or"  # Output: 7
echo "XOR: $xor" # Output: 6

```

For & → they look for same bit positions. For | → they look for either positions, for ^, they see if there are any common positions. Here, for &, they look for 0001, which is bit for 1, for |, they look for 0111, bit for 7, for ^, they look for 0110, where no common element present.

```

echo "Enter your numbers:"
read a b c
let sum='a*(b+c)'
let sum++
echo "The sum is $sum"

```

```

gayat@GSP-HP:~/SummerInternship$ ./script1.sh
Enter your numbers:
4 5 6
The sum is 45
gayat@GSP-HP:~/SummerInternship$

```

Here, first, I assigned an arithmetic expression using a, b and c and then assigned an increment to the sum, being the variable assigned to the arithmetic expression and then I echoed it to get the result, so $4(5+6)=44=\text{sum}$, but `sum++` makes it 45.

- `pushd +n` takes the nth directory in the stack (starting with 0), rotates it to the top, and `cds` to it.
- `pushd` without arguments, instead of complaining, swaps the two top directories on the stack and `cds` to the new top.
- `popd +n` takes the nth directory in the stack and just deletes it

```

gayat@GSP-HP:~$ pushd S
SummerInternship/ snap/
gayat@GSP-HP:~$ pushd SummerInternship/
~/SummerInternship ~
gayat@GSP-HP:~/SummerInternship$ pushd te
test1/ test2/ test2705/ test3/
gayat@GSP-HP:~/SummerInternship$ pushd test2705/
~/SummerInternship/test2705 ~/SummerInternship ~
gayat@GSP-HP:~/SummerInternship/test2705$ popd
~/SummerInternship ~
gayat@GSP-HP:~/SummerInternship$ popd
~
gayat@GSP-HP:~$ popd
bash: popd: directory stack empty

```

```

gayat@GSP-HP:~/SummerInternship/test1$ pushd
~/SummerInternship ~/SummerInternship/test1 ~
gayat@GSP-HP:~/SummerInternship$ popd +2
~/SummerInternship ~/SummerInternship/test1
gayat@GSP-HP:~/SummerInternship$ popd +1
~/SummerInternship

```

ARITHMETIC for LOOP

```

for (( init; condition; increment ))
do
    # commands
done

```

The first 3 inside the double parentheses are arithmetic expressions, and the last is a set of statements as in a standard loop.

Init → something as a starting value ($i=0$), when evaluated, comes true, continues the loop.

Condition → continues while this condition is said to be true. The loop continues until this condition is satisfied.

Increment → The increment is how i changes with time ($i++$)



20250617-1129-21.5
303713.mp4

→ the video being another way of printing the same thing!

```

gayat@GSP-HP:~/SummerInternship$ ./echo.sh
for (( i=1; i <= 12 ; i++ ))
do
    for (( j=1; j <= 12 ; j++ ))
    do
        echo -ne "$(( j * i ))\t"
    done
    echo
done

```

1	2	3	4	5	6	7	8	9	10	11	12
2	4	6	8	10	12	14	16	18	20	22	24
3	6	9	12	15	18	21	24	27	30	33	36
4	8	12	16	20	24	28	32	36	40	44	48
5	10	15	20	25	30	35	40	45	50	55	60
6	12	18	24	30	36	42	48	54	60	66	72
7	14	21	28	35	42	49	56	63	70	77	84
8	16	24	32	40	48	56	64	72	80	88	96
9	18	27	36	45	54	63	72	81	90	99	108
10	20	30	40	50	60	70	80	90	100	110	120
11	22	33	44	55	66	77	88	99	110	121	132
12	24	36	48	60	72	84	96	108	120	132	144

```

for (( i=1; i <= 5 ; i++ ))
do
    for (( j=1; j <= 10 ; j++ ))
    do
        echo -ne "$(( $i * $j ))\t"
    done
    echo
done

```

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50

ARRAYS:

- An array is like a series of slots that hold values.
- Each slot is known as an element, and each element can be accessed via a numerical index.
- An array element can contain a string or a number, and you can use it just like any other variable.
- The indices for arrays start at 0 and continue up to a very large number.
- To explicitly create an empty array, you can use the `a` option to declare.
- Any attributes that you set for the array with `declare` (e.g., the read-only attribute) apply to the entire array.

HOW TO ASSIGN VALUES TO THE ARRAY ELEMENTS:

```

gayat@GSP-HP:~/SummerInternship$ names[1]=surprise
gayat@GSP-HP:~/SummerInternship$ names[4]=Hello

```

We didn't have to specify the indices in numerical order! Even the indices aren't necessary, you could just type in

```
names=(surprise hello)
```

and the bash will arrange the array elements from element 0 to large numbers.

```
names=(hatter [5]=duchess alice)
```

then, bash assigns `hatter` to element 0, `duchess` to element 5, and `alice` to element 6.

How to print the element in an array →

```

gayat@GSP-HP:~/SummerInternship$ echo ${names[1]}
surprise

names=( hello hi niece to talk howdy )

for i in "${names[@]"; do
    echo $i
done

```

To find out the length of any element in the array, you can use `${#array[i]}`. Similarly, to find out how many values there are in the array, use `*` or `@` as the index. So, for `names=(hatter [5]=duchess alice)`, `${#names[5]}` has the value 7, and `${#names[@]}` has the value 3.

Reassigning to an existing array with a compound array statement replaces the old array with the new one. All of the old values are lost, even if they were at different indices to the new elements.

You can destroy any element or the entire array by using the `unset` built-in.

However, unlike assignment, if you don't specify an index the entire array is unset, not just element 0. You can explicitly specify unsetting the entire array by using `*` or `@` as the index.

```
for i in $(cut -f 1,3 -d: /etc/passwd) ; do
    array[${i#*:}]=${i%:*}
done
echo "User ID $1 is ${array[$1]}."
echo "There are currently ${#array[@]} user accounts"

gayat@GSP-HP:~/SummerInternship$ ./echo.sh
User ID is root.
There are currently 28 user accounts
gayat@GSP-HP:~/SummerInternship$
```

Line	Meaning
<code>cut -f 1,3 -d:</code>	Extracts username and UID from <code>/etc/passwd</code>
<code>array[UID]=username</code>	Builds a map of UID to username
<code>\${array[\$1]}</code>	Looks up username for given UID
<code>\${#array[@]}</code>	Counts user accounts